

SISTEMA ALQUILER QUADS

Curso 2025/2026

Autores:

Imad Habib Jali

901421@unizar.es

Jorge Bellido Lobera

903080@unizar.es

Martes A

Índice

0 - Resumen.....	3
1 - Catálogo de requisitos.....	4
REQUISITOS FUNCIONALES.....	4
REQUISITOS NO FUNCIONALES.....	5
2- Definición de casos de uso.....	6
3 - Modelado estático de la aplicación.....	9
4 - Modelado dinámico de la aplicación.....	9
5 - Diagrama de flujo.....	13
6 - Prototipo.....	14
7- Modelo de diseño.....	19
8. Conclusiones y Mejoras.....	27

0 - Resumen

El proyecto consiste en el desarrollo de una solución móvil robusta bajo la plataforma Android, destinada a la gestión integral de un negocio de alquiler de vehículos tipo quad. La aplicación permite administrar de forma eficiente el inventario de vehículos (especificaciones técnicas, tipos de plaza y tarificación) y el ciclo de vida de las reservas de clientes. Entre sus funcionalidades principales destaca la lógica de comprobación de disponibilidad en tiempo real, que garantiza la integridad de la planificación al evitar solapamientos de fechas, así como un módulo de comunicación profesional que permite el envío de confirmaciones detalladas a través de servicios de mensajería externos.

Arquitectónicamente, la aplicación se sustenta en el patrón de diseño Model-View-ViewModel (MVVM), lo que garantiza una separación clara entre la interfaz de usuario y la lógica de negocio, facilitando el mantenimiento y la escalabilidad del código. Para la persistencia de datos local se ha empleado la librería Room (SQLite), gestionando una estructura relacional que incluye relaciones de muchos a muchos mediante tablas intermedias para la asignación de equipamiento adicional (casco). Además, se han integrado patrones de diseño fundamentales como el patrón Bridge, aplicado para desacoplar la lógica de envío de mensajes (SMS y WhatsApp) de la capa de presentación, y el patrón Singleton para la gestión centralizada de la base de datos.

La fiabilidad del sistema ha sido validada mediante un exhaustivo plan de pruebas que abarca tests unitarios de caja negra, asegurando que el repositorio actúe como un filtro contra datos inconsistentes mediante el uso de expresiones regulares y validaciones de integridad. Asimismo, se han ejecutado pruebas de volumen para verificar la fluidez de la interfaz ante cargas de hasta 20.000 registros y pruebas de sobrecarga para determinar los límites de capacidad de almacenamiento de caracteres.

El resultado es una herramienta con una estética cuidada en tonos pastel y una experiencia de usuario optimizada, capaz de responder con solvencia a las demandas operativas de una empresa del sector.

1 - Catálogo de requisitos

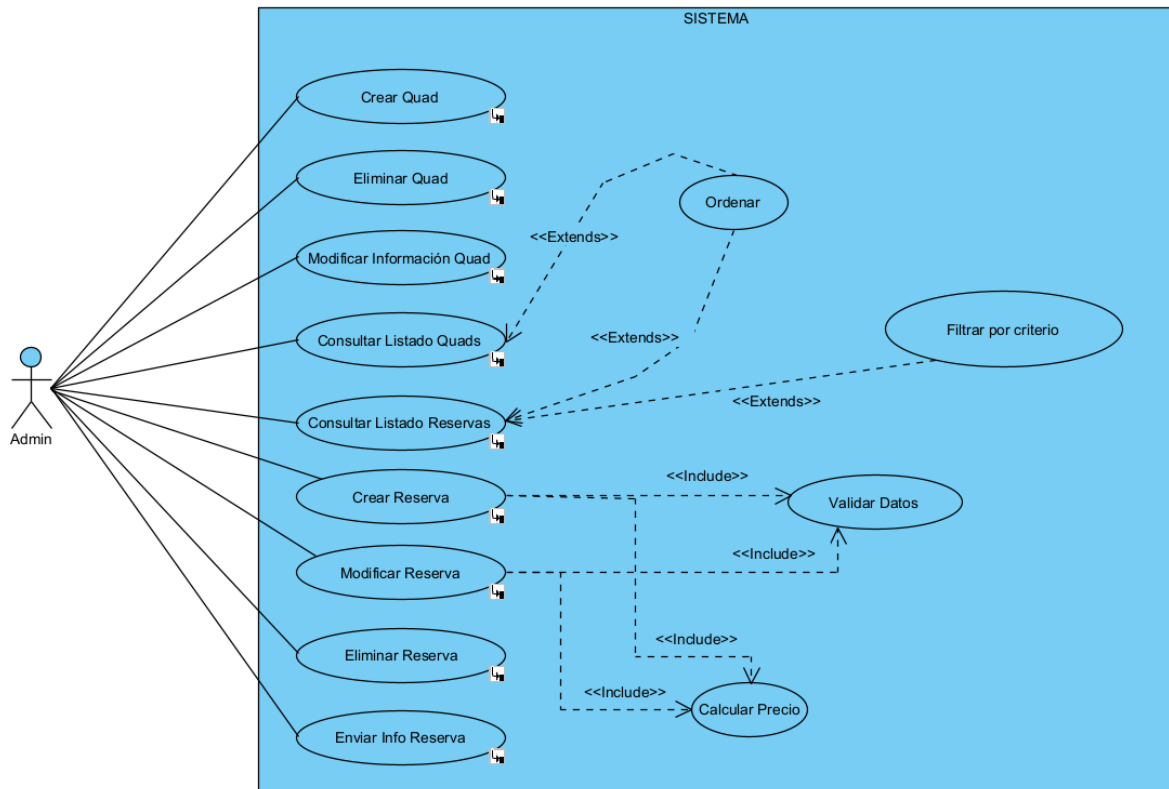
REQUISITOS FUNCIONALES	
RF 1	La aplicación debe permitir la creación de quads, para cada quad se debe almacenar la matrícula, tipo de quad, precio en euros del alquiler por día, y una breve descripción en texto libre que puede ser información sobre la marca, modelo, color o cualquier otra característica que se considere relevante. Adicionalmente, el sistema asignará automáticamente un identificador único interno a cada vehículo registrado para gestionar su ciclo de vida de forma independiente a su matrícula, asegurando así la adaptabilidad del sistema ante posibles cambios en la normativa o en el tipo de flota.
RF 2	La aplicación debe permitir la consulta de un listado de quads previamente creados, pudiéndose ordenar por matrícula, tipo o precio.
RF 3	La aplicación debe permitir la modificación de información relativa a los quads previamente creados.
RF 4	La aplicación debe permitir dar de baja un quad. El elemento dejará de estar disponible para nuevas operaciones, pero su información se conservará mediante un borrado lógico para mantener el historial contable
RF 5	La aplicación debe permitir la creación de una reserva, la reserva consta de un nombre de cliente, el número móvil del cliente, la fecha de recogida y devolución, y una selección de quads, indicando también si se va a necesitar casco para su conducción.
RF 6	La aplicación debe permitir la comprobación de que la reserva es válida, es decir, que la fecha de devolución es igual o posterior a la del día de recogida, que no se producen solapes en la reserva de quads y que no se supera el número de cascos solicitados para un quad monoplace
RF 7	La aplicación debe permitir la consulta de un listado de las reservas previamente creadas, pudiéndose ordenar por nombre de cliente, número de móvil, fecha de recogida, o fecha de devolución.
RF 8	La aplicación debe permitir la modificación de las reservas previamente creadas.
RF 9	La aplicación debe permitir dar de baja una reserva. El elemento dejará de estar disponible para nuevas operaciones, pero su información se conservará mediante un borrado lógico para mantener el historial contable
RF 10	La aplicación debe permitir la cálculo del precio total de una reserva, el precio total se debería calcular de forma automática en la creación de la reserva, o en la modificación de la reserva si se realiza una

	selección distinta de quads.
RF 11	La aplicación debe permitir el envío de la información de la reserva al móvil del cliente cuando el administrador lo considere oportuno.
RF 12	El sistema debe mantener el precio total de una reserva aunque se modifique posteriormente el precio del alquiler por día de un quad.
RF 13	El sistema debe permitir visualizar la lista de reservas de acuerdo con los siguientes criterios: 1. Reservas previstas: reservas cuya fecha de recogida no se ha alcanzado todavía. 2. Reservas vigentes: reservas cuya fecha de recogida ha sido alcanzada, pero cuya fecha de devolución no ha vencido todavía. 3. Reservas caducadas: reservas cuya fecha de devolución se ha sobrepasado. 4. Todas las reservas, con independencia de sus fechas de recogida y devolución. Este será el modo de visualización del listado de reservas por defecto al iniciarse el sistema.

REQUISITOS NO FUNCIONALES

RNF 1	El sistema debe tener capacidad para almacenar al menos 100 quads y 20000 reservas.
RNF 2	La aplicación deberá funcionar en dispositivos móviles con sistema operativo Android 8.0 (API 26) o superior

2- Definición de casos de uso



Precondición Global / Entorno de Ejecución: Todos los casos de uso y flujos descritos en este documento asumen como precondición global que el actor (administrador) interactúa con el sistema a través de la interfaz gráfica de una aplicación móvil ejecutada en el sistema operativo Android.

CU-1. Caso de uso "Crear Quad":

Flujo principal:

- 1) El caso de uso comienza cuando el administrador quiere añadir un nuevo quad.
- 2) A continuación introduce la matrícula, tipo de quad, precio en euros del alquiler por día, y una breve descripción en texto libre que puede ser información sobre la marca, modelo, color o cualquier otra característica que se considere relevante.
- 3) El sistema verifica que exista espacio de almacenamiento disponible, garantizando el registro de hasta 100 quads.
- 4) El quad se añade con su respectiva información al sistema.

CU-2. Caso de uso "Eliminar Quad":

Flujo principal:

- 1) El caso de uso comienza cuando el administrador quiere eliminar un quad existente en el sistema.
- 2) A continuación busca el quad deseado y lo elimina.
- 3) El quad es eliminado junto con su respectiva información del sistema.

CU-3. Caso de uso "Modificar Información Quad":

- 1) El caso de uso comienza cuando el administrador quiere modificar la información de un quad.
- 2) A continuación modifica el dato que se considere.
- 3) La información del quad modificada se actualiza en el sistema

CU-4. Caso de uso "Consultar Listado Quads":

Flujo principal:

- 1) El caso de uso comienza cuando el administrador quiere consultar el listado de quads.
- 2) El sistema recupera la información y muestra el listado de quads previamente creados en su formato y orden por defecto.

Flujos alternativos:

A. Ordenar el listado (Extensión):

- 1) Tras visualizar el listado (paso 2 del flujo principal), el administrador elige ordenar la vista especificando uno de los siguientes criterios: matrícula, tipo o precio.
- 2) El sistema procesa la solicitud y actualiza la pantalla mostrando el listado de quads con la nueva ordenación elegida.

CU-5. Caso de uso "Consultar Listado Reservas":

Flujo principal:

- 1) El caso de uso comienza cuando el administrador quiere consultar el listado de reservas.
- 2) El sistema recupera la información y muestra el listado de reservas previamente creadas en su formato y orden por defecto.

Flujos alternativos:

A. El Administrador decide Ordenar el listado:

- 1) El Administrador selecciona una de las opciones de ordenación disponibles: nombre de cliente, número de móvil, fecha de recogida o fecha de devolución.
- 2) El sistema actualiza y muestra el listado ordenado según el criterio elegido.

B. El Administrador decide Filtrar el listado:

- 1) El Administrador selecciona uno de los siguientes filtros de estado:
Reservas previstas: Cuya fecha de recogida no se ha alcanzado.
Reservas vigentes: Cuya fecha de recogida ha sido alcanzada, pero la de devolución no ha vencido.
Reservas caducadas: Cuya fecha de devolución se ha sobrepasado.
Reservas totales
- 2) El sistema actualiza y muestra el listado mostrando únicamente las reservas que coinciden con el filtro seleccionado.

CU-6. Caso de uso "Crear Reserva":

Flujo principal:

- 1) El caso de uso comienza cuando el administrador quiere hacer una nueva reserva.
- 2) A continuación introduce un nombre, el número móvil, la fecha de recogida y devolución, y una selección de quads, indicando también número de cascos.
- 3) El sistema comprueba que la reserva es válida, es decir, que la fecha de devolución es igual o posterior a la del día de recogida, que no se producen solapes en la reserva de quads y que no se supera el número de cascos solicitados para un quad monoplace.
- 4) El sistema verifica que la base de datos no haya superado el límite de 20.000 reservas operativas.
- 5) El sistema calcula el importe de la reserva (Incluye CU-8: Calcular Precio).
- 6) La reserva se añade con su respectiva información al sistema.

CU-7. Caso de uso "Modificar Información Reserva":

- 1) El caso de uso comienza cuando el administrador quiere modificar la información de una reserva existente.
- 2) A continuación modifica los datos que se considere.
- 3) El sistema comprueba que la reserva es válida, es decir, que la fecha de devolución es igual o posterior a la del día de recogida, que no se producen solapes en la reserva de quads y que no se supera el número de cascos solicitados para un quad monoplace.
- 4) El sistema calcula el importe de la reserva (Incluye CU-8: Calcular Precio), respetando al cliente(RF12).
- 5) La información de la reserva modificada se actualiza en el sistema.

CU-8. Caso de uso "Eliminar Reserva":

Flujo principal:

- 1) El caso de uso comienza cuando el administrador quieren eliminar una reserva existente en el sistema.
- 2) A continuación busca la reserva deseada y la elimina.
- 3) La reserva es eliminada con su respectiva información del sistema.

CU-9. Caso de uso "Enviar Información":

Flujo principal:

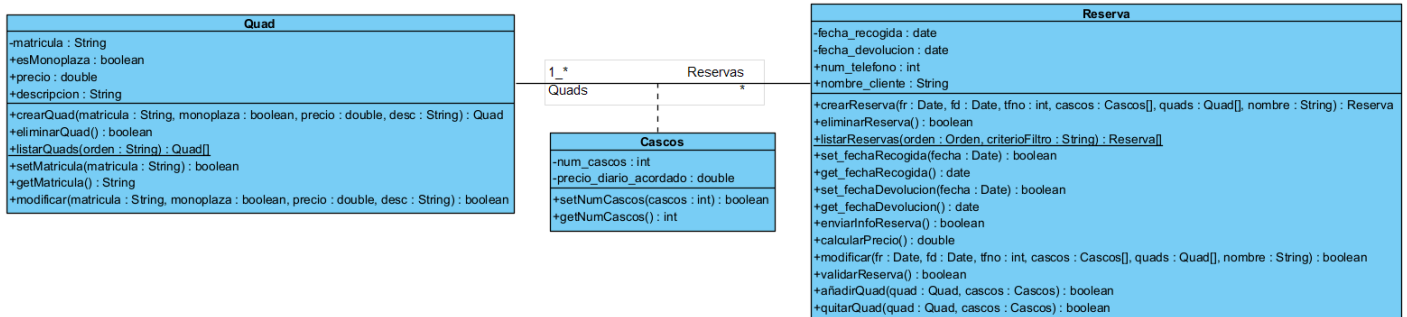
- 1) El caso de uso comienza cuando el administrador considera oportuno enviar dicha información al cliente.
- 2) El sistema envía la información de la reserva al móvil del cliente.

CU-10. Caso de uso "Calcular Precio" (Include):

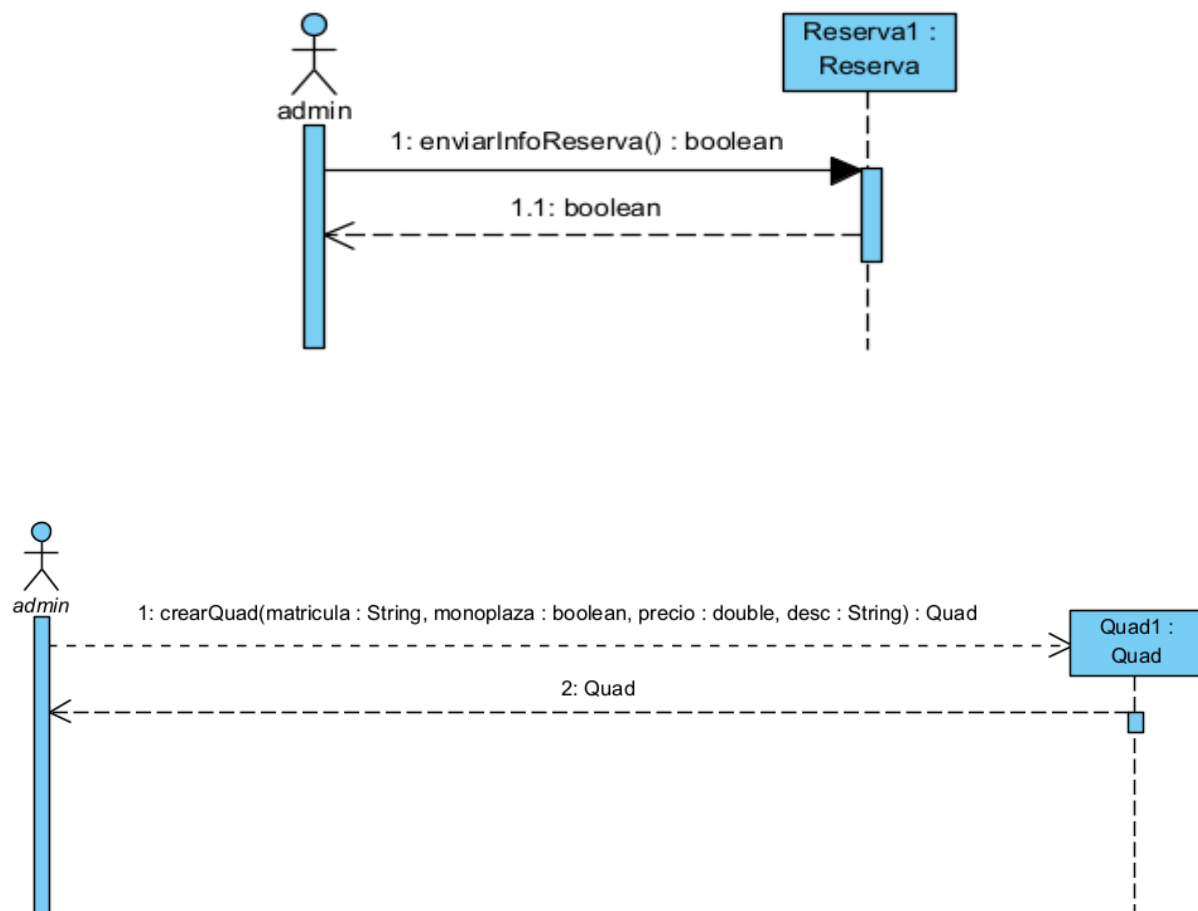
Flujo principal:

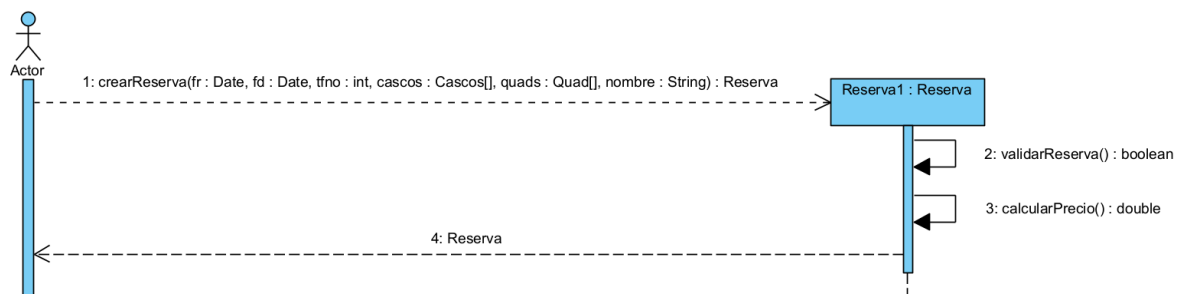
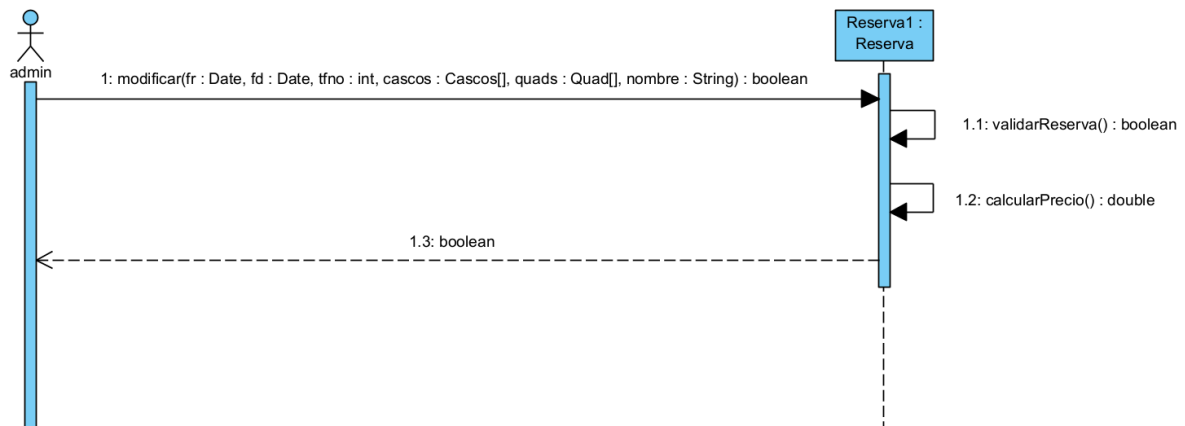
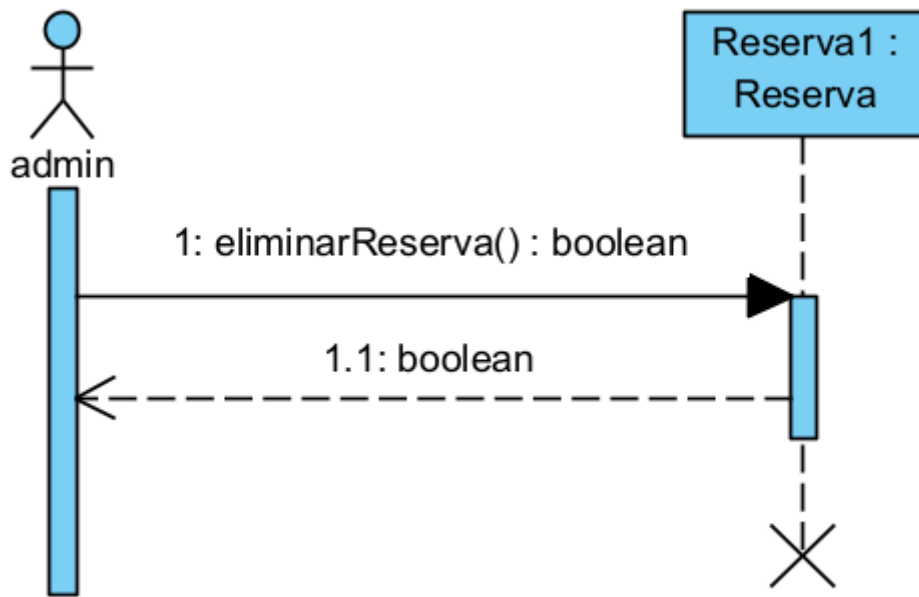
- 1) El caso de uso comienza cuando un proceso requiere determinar un importe económico.
- 2) El sistema consulta las tarifas vigentes en la base de datos para los quads seleccionados.
- 3) El sistema calcula el número de días de alquiler.
- 4) El sistema multiplica los días por el precio acordado por quad por día.
- 5) El sistema devuelve el resultado al caso de uso de origen.

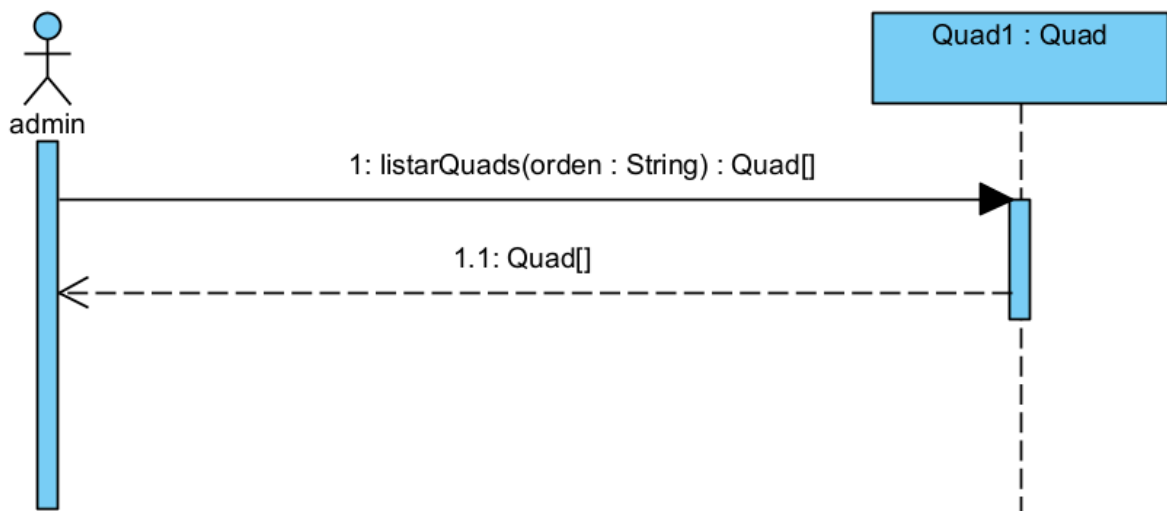
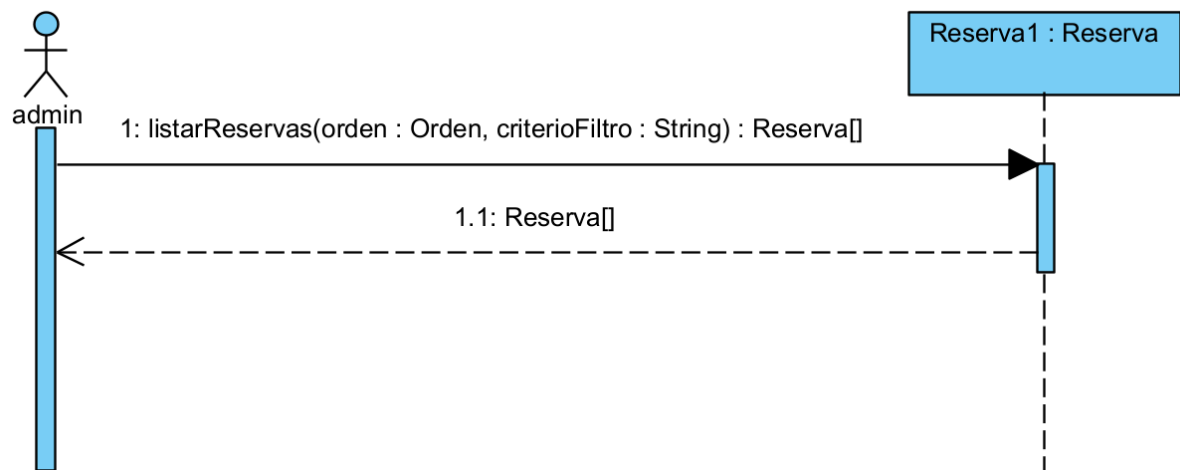
3 - Modelado estático de la aplicación

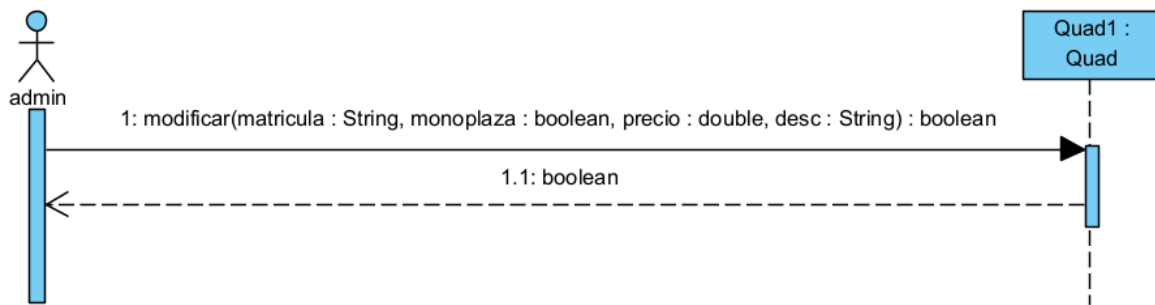
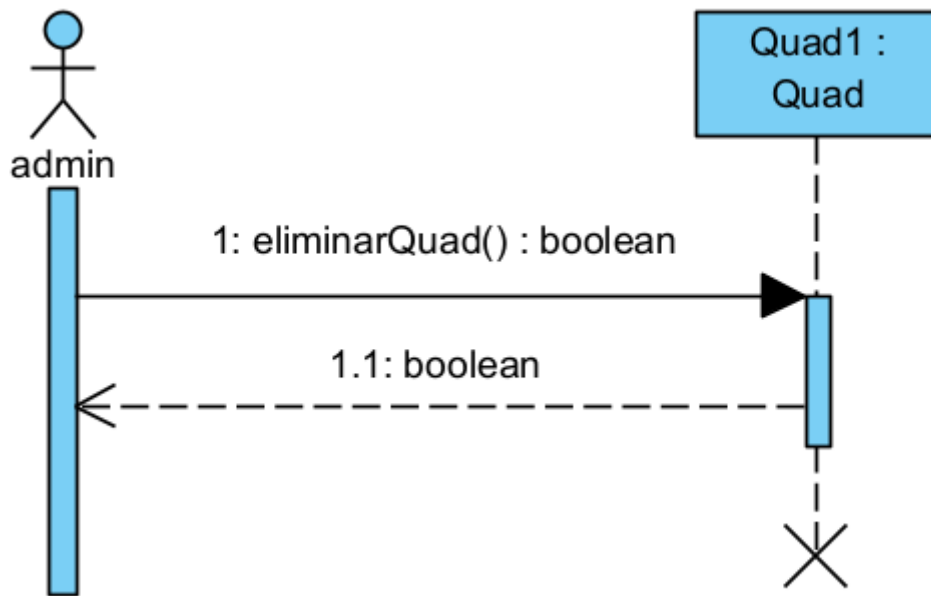


4 - Modelado dinámico de la aplicación

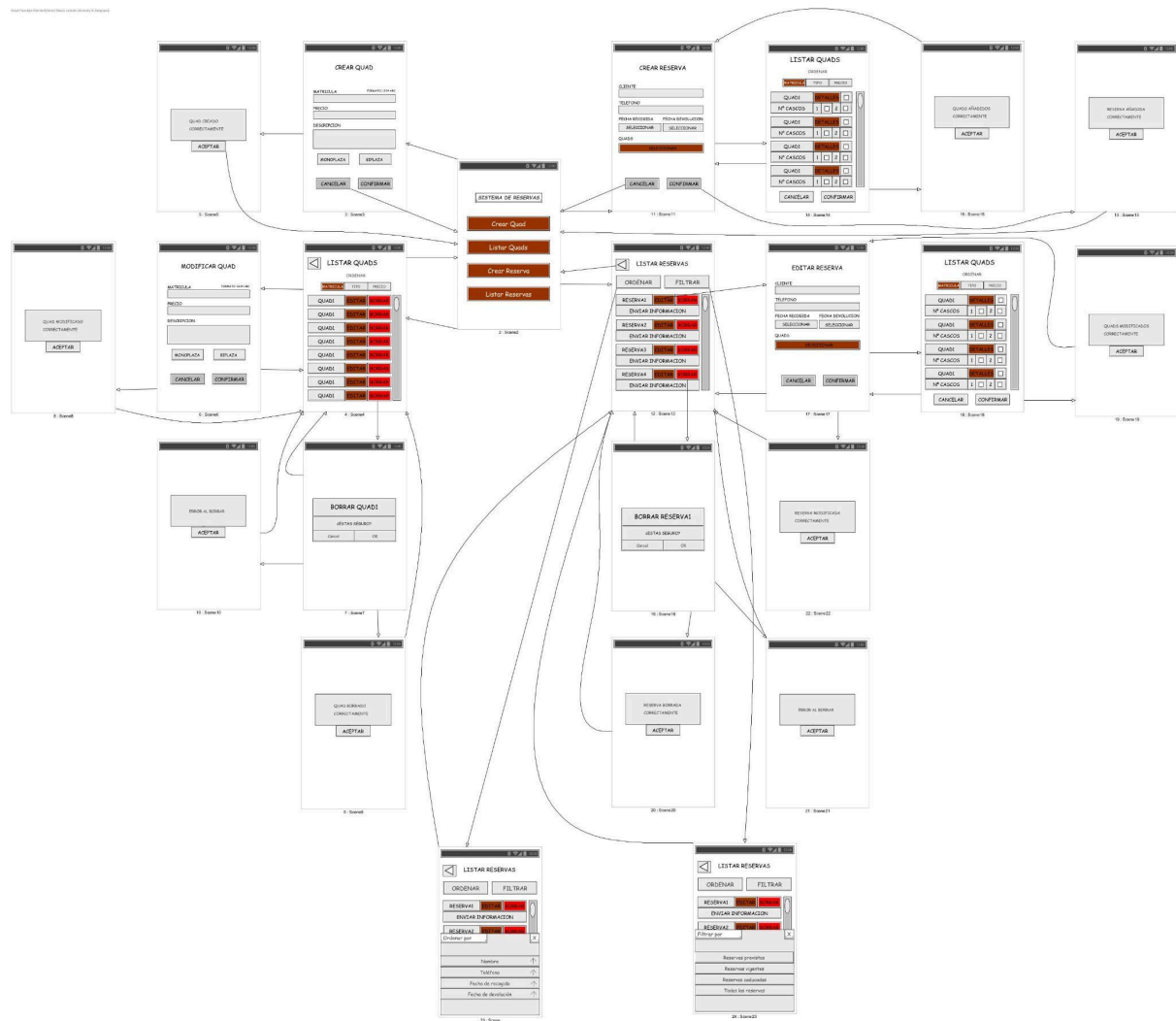




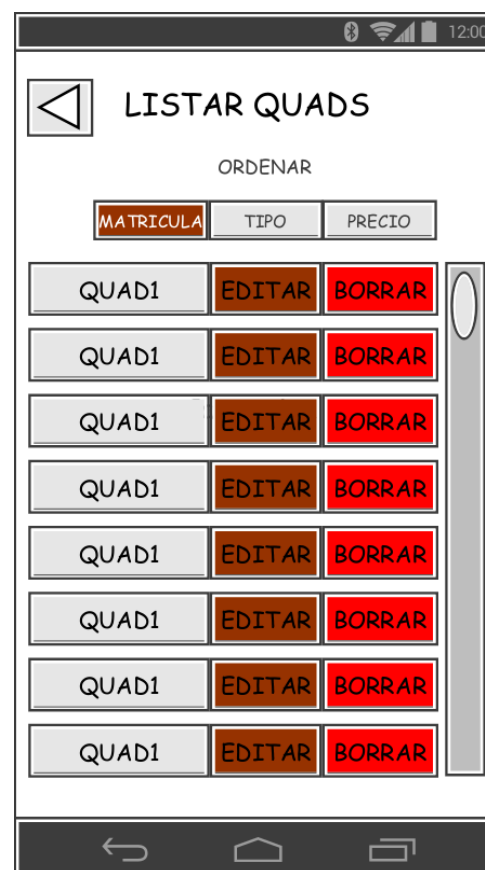
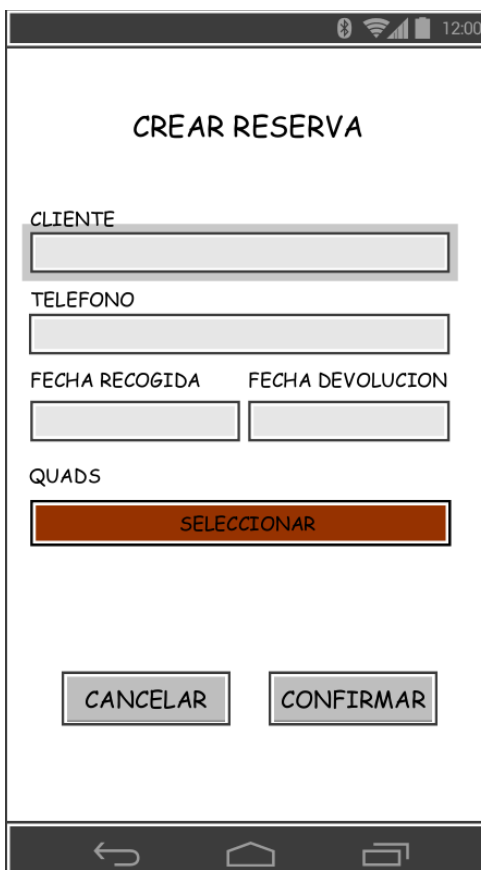
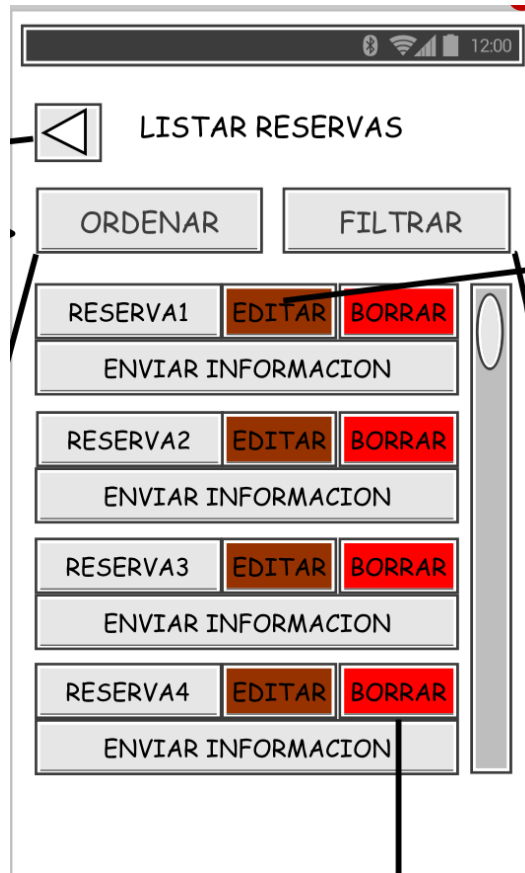


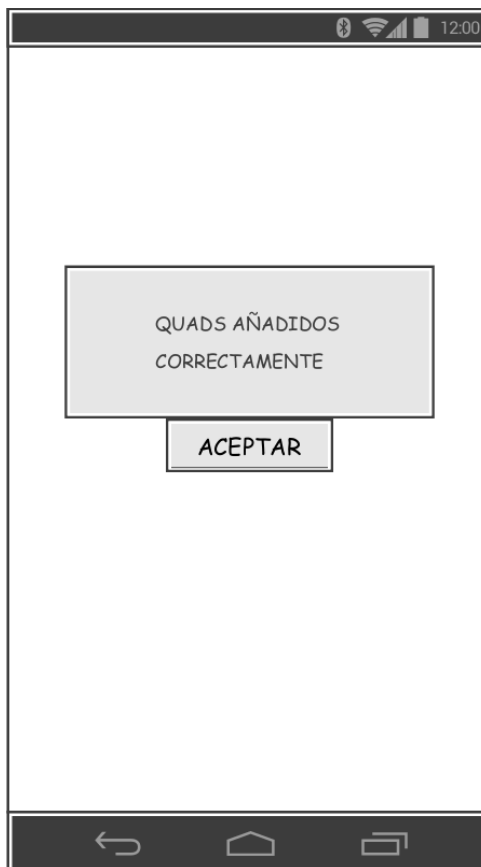
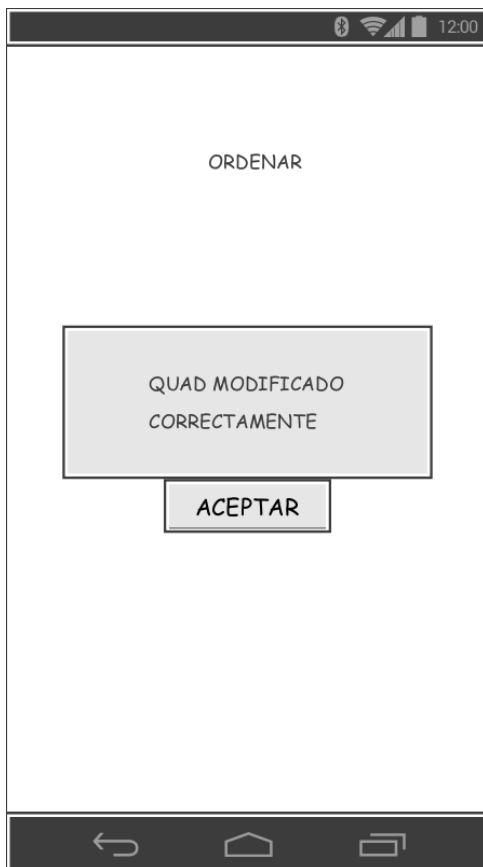


Journal Pre-proof



6 - Prototipo





ERROR AL BORRAR

ACEPTAR

RESERVA BORRADA
CORRECTAMENTE

ACEPTAR

EDITAR RESERVA

CLIENTE

TELEFONO

FECHA RECOGIDA FECHA DEVOLUCION

QUADS

CANCELAR

CONFIRMAR

LISTAR QUADS

ORDENAR

MATRICULA TIPO PRECIO

QUAD1	DETALLES	
Nº CASCOS	1	2
QUAD1	DETALLES	
Nº CASCOS	1	2
QUAD1	DETALLES	
Nº CASCOS	1	2
QUAD1	DETALLES	
Nº CASCOS	1	2

CANCELAR

CONFIRMAR

12:00

CREAR QUAD

MATRICULA FORMATO: 1234 ABC

PRECIO

DESCRIPCION

MONOPLAZA BIPLAZA

CANCELAR CONFIRMAR

← ↗ ↶

12:00

QUADS MODIFICADOS
CORRECTAMENTE

ACEPTAR

← ↗ ↶

12:00

RESERVA MODIFICADA
CORRECTAMENTE

ACEPTAR

← ↗ ↶

12:00

INFORMACION ENVIADA
CORRECTAMENTE

ACEPTAR

← ↗ ↶

12:00

MODIFICAR QUAD

MATRICULA

FORMATO: 1234 ABC

PRECIO

DESCRIPCION

MONOPLAZA

BIPLAZA

CANCELAR

CONFIRMAR

12:00

QUAD CREADO
CORRECTAMENTE

ACEPTAR

12:00

BORRAR QUAD1

¿ESTAS SEGURO?

Cancel

OK

12:00

ERROR AL BORRAR

ACEPTAR



7- Modelo de diseño

DIAGRAMAS DE COMPONENTES Y DESPLIEGUE

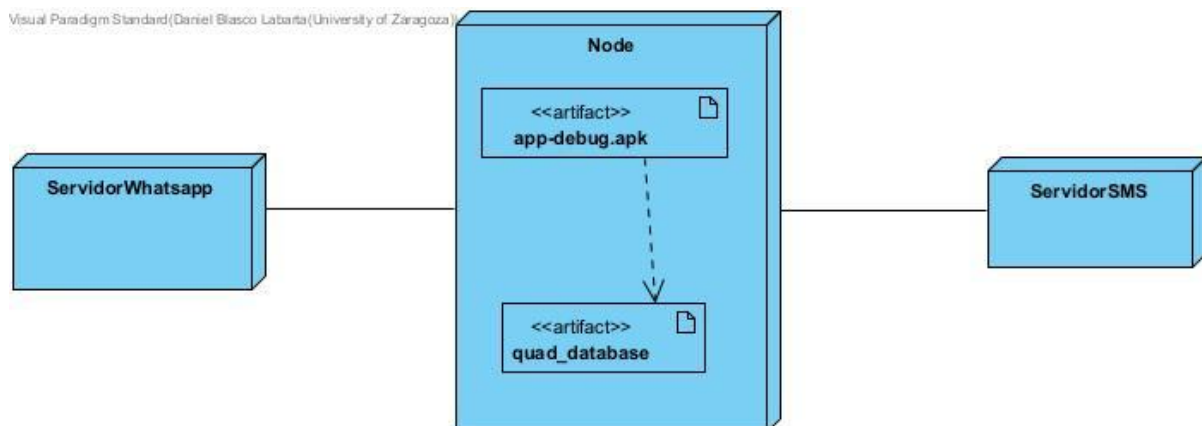
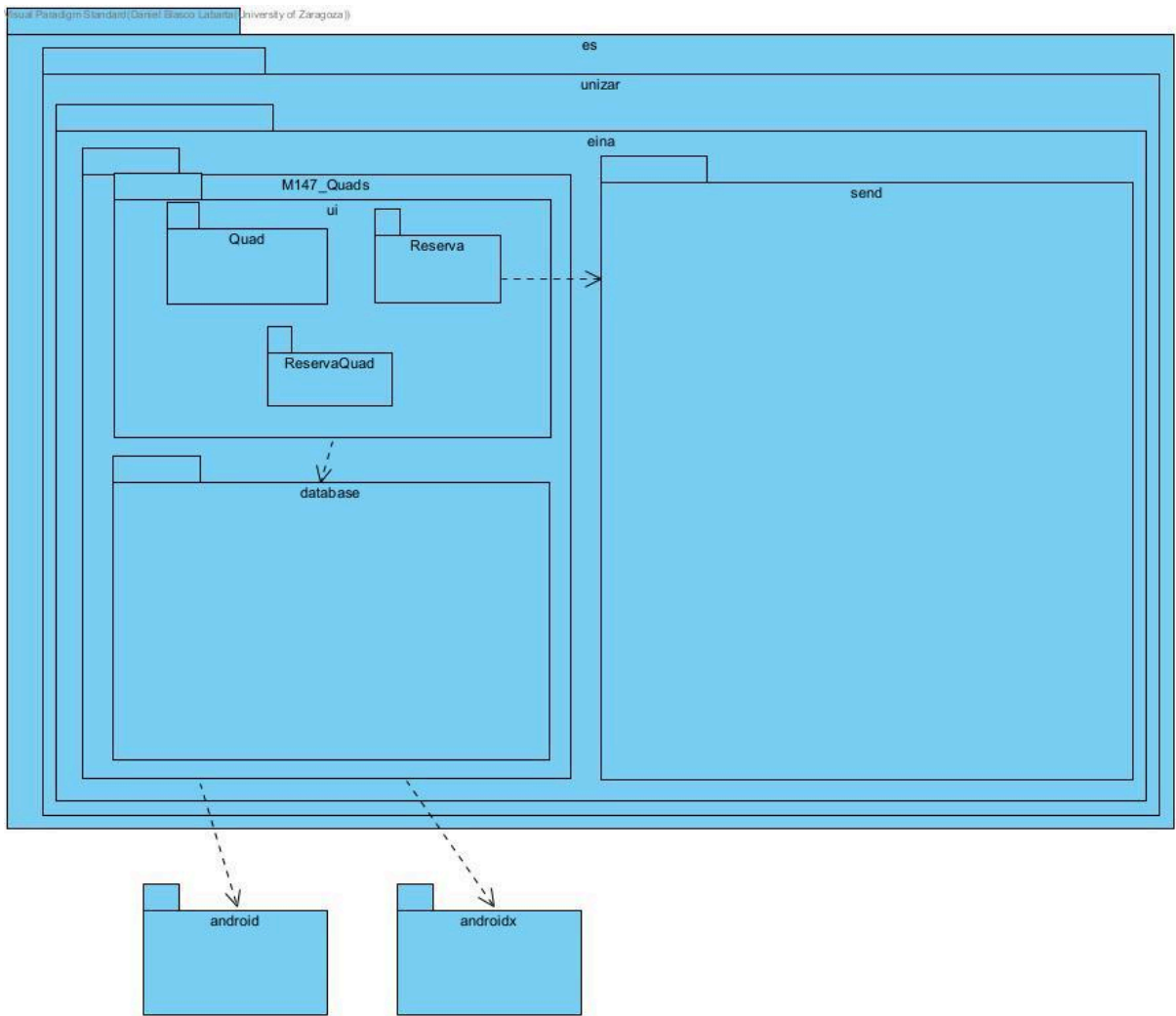


DIAGRAMA DE PAQUETES



El diagrama ilustra la organización jerárquica y modular de la aplicación. Se observa una estructura de capas claramente diferenciada, diseñada para maximizar la separación de responsabilidades y facilitar el mantenimiento del sistema.

Visual Paradigm Standard (Daniel Blasco Loboña/University of Zaragoza)



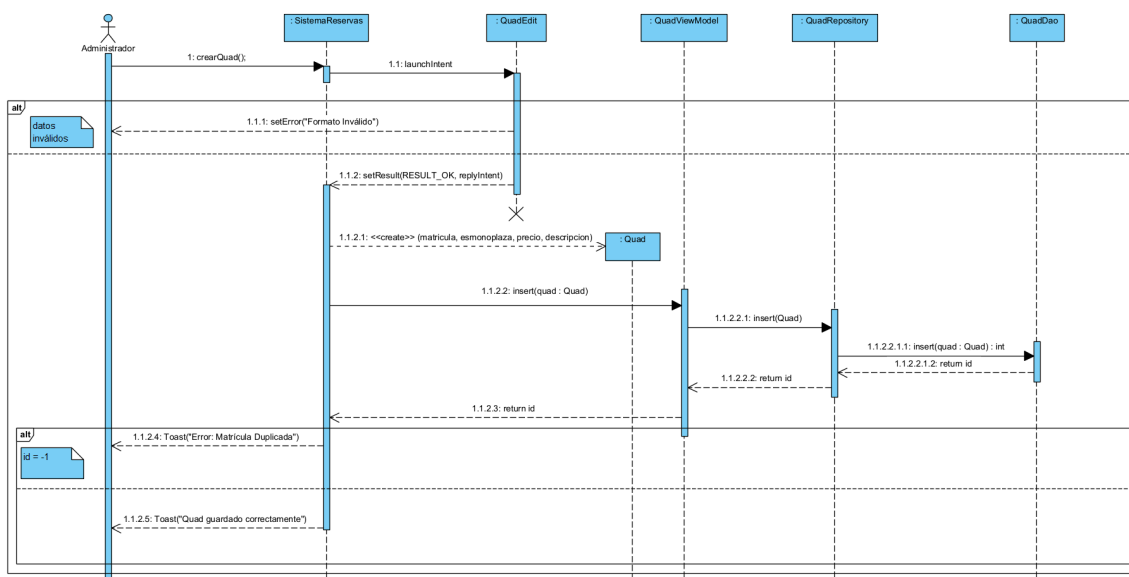


Figura 7.5 Creación de un quad

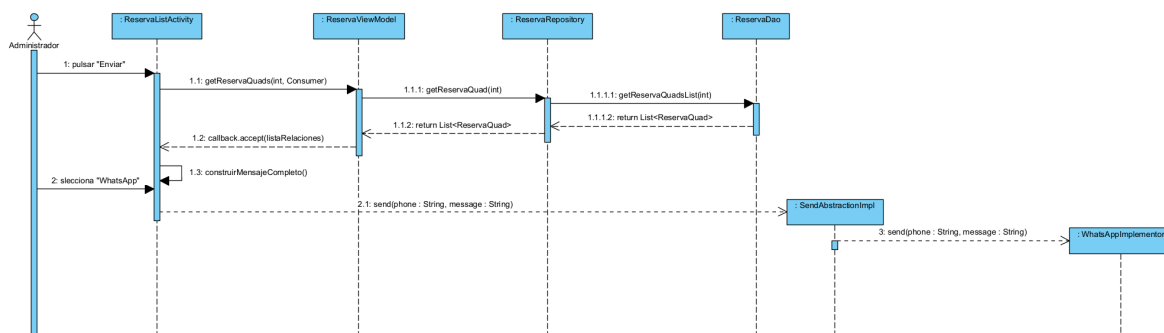


Figura 7.6 Enviar información de reserva

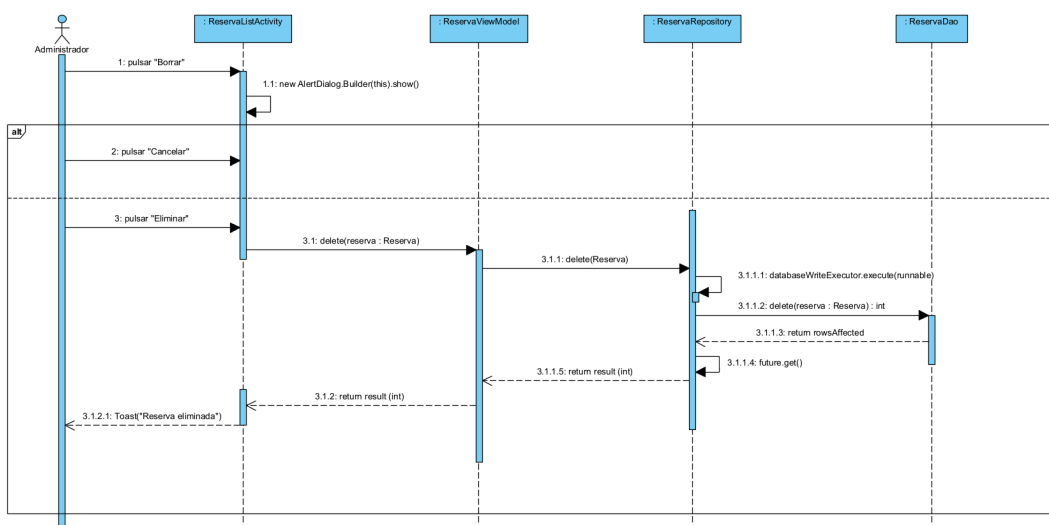


Figura 7.7 Eliminar reserva

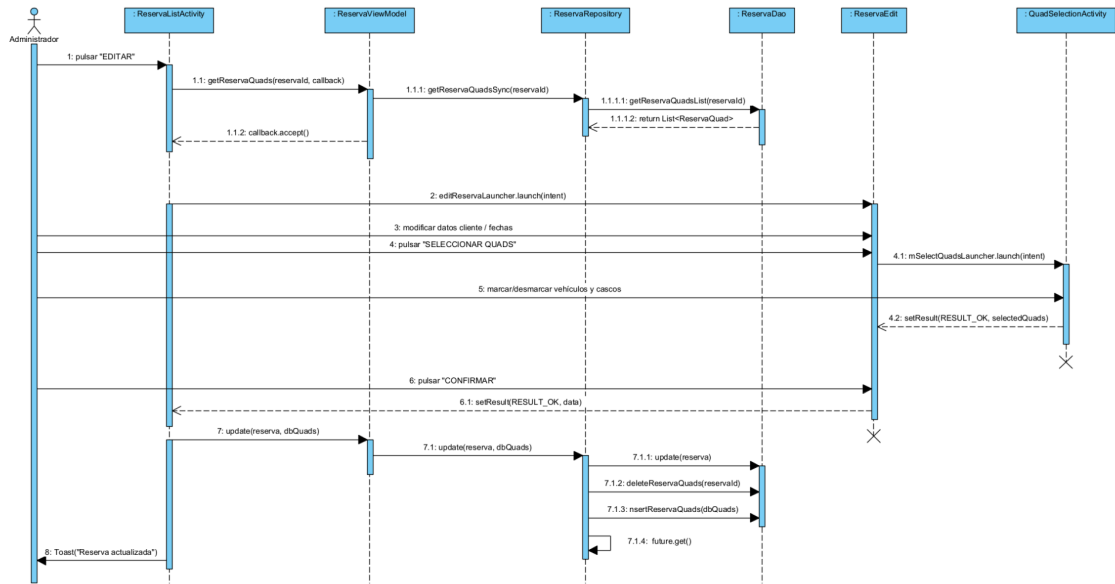


Figura 7.8 Editar Reserva

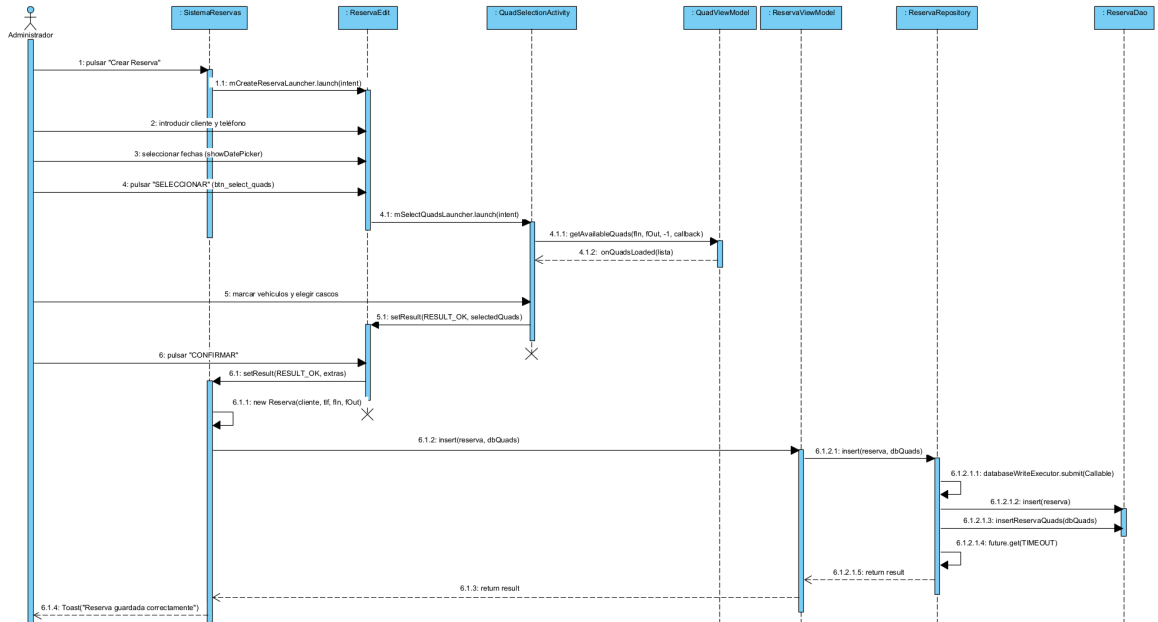


Figura 7.9 Crear Reserva

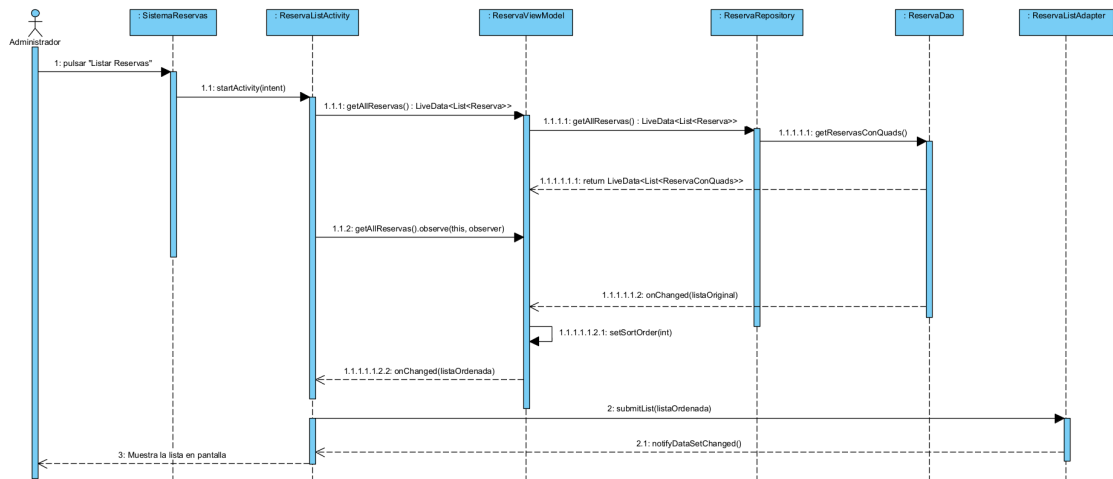


Figura 7.10 Listar reservas

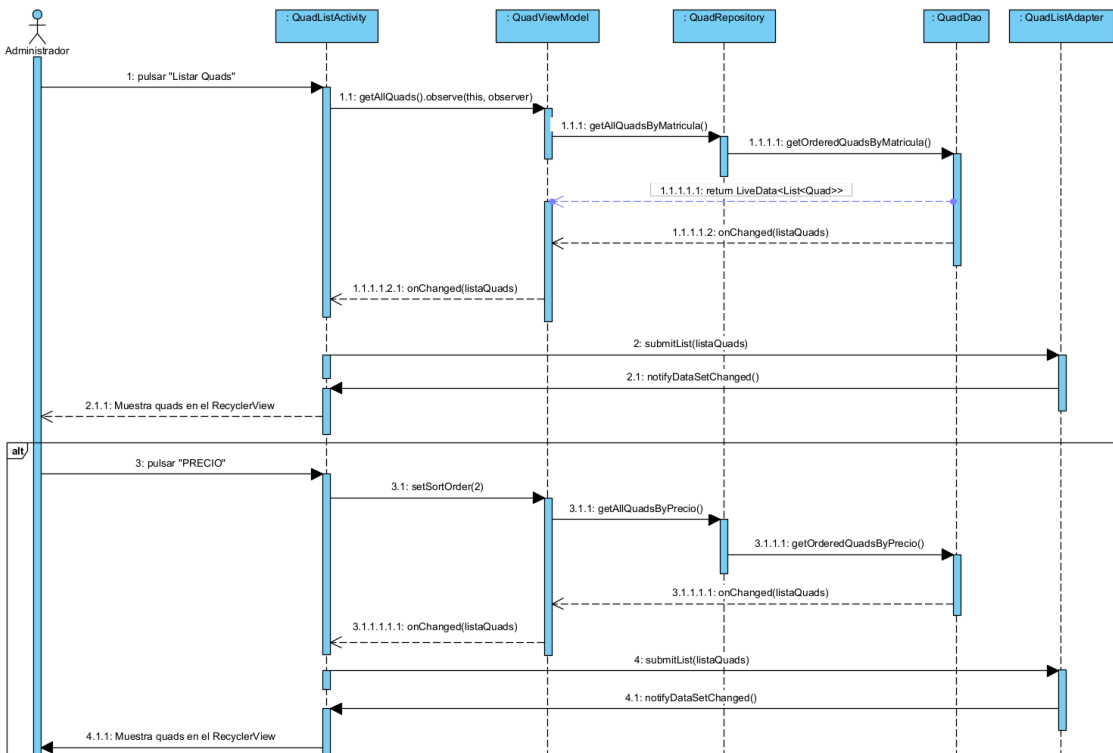


Figura 7.11 Listar Quads

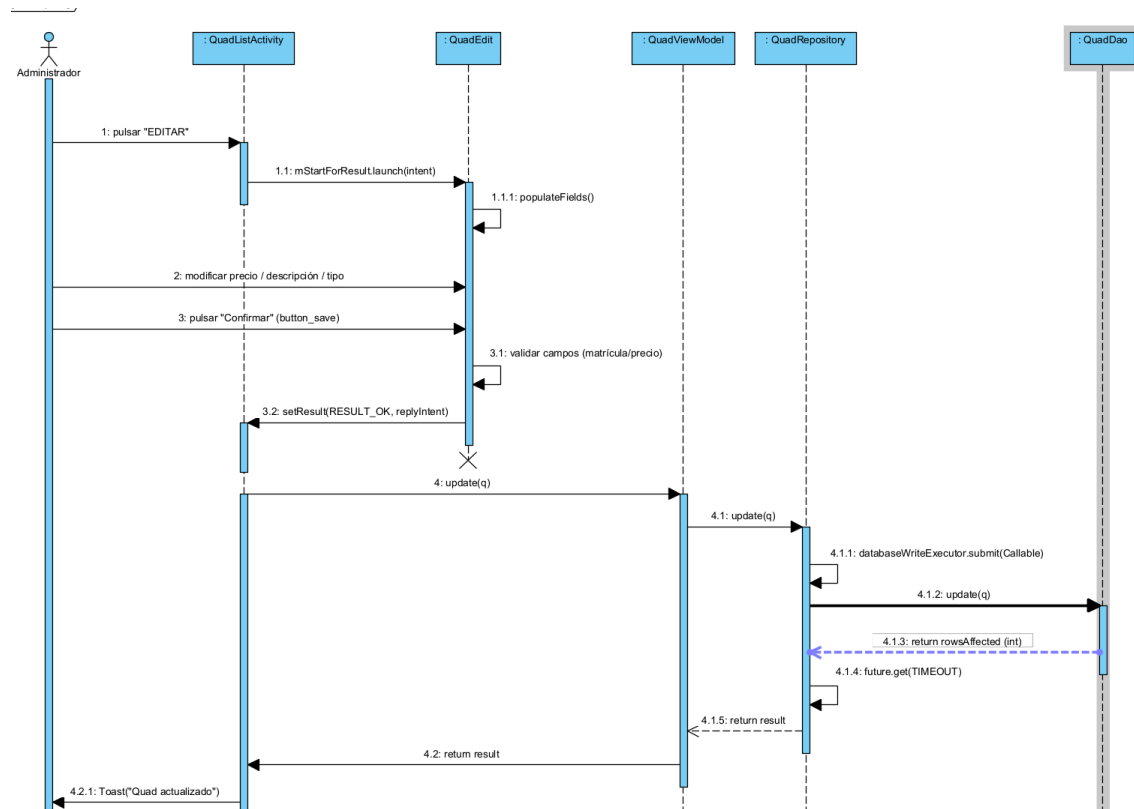


Figura 7.12 Editar Quad

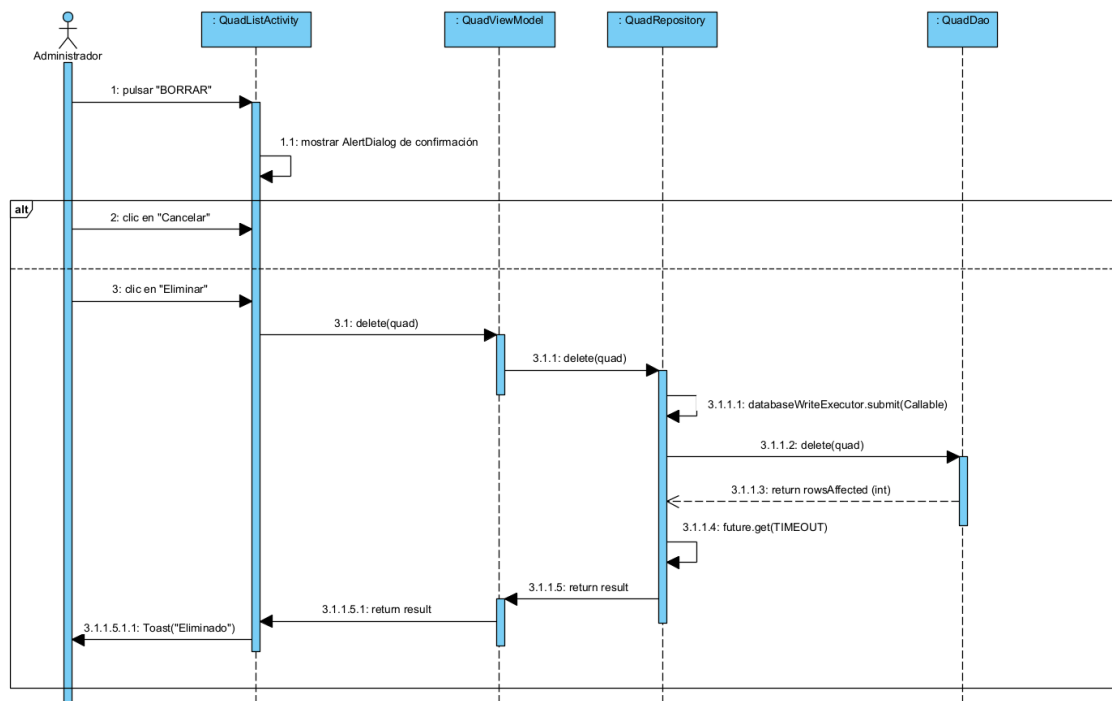
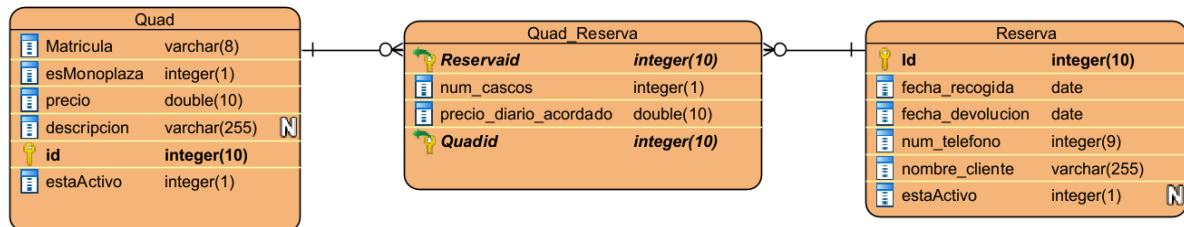


Figura 7.13 Borrar Quad

MODELO LÓGICO

La base de datos que se va a utilizar para almacenar la información sobre los quads y las reservas es de tipo relacional. Utilizando como punto de partida el modelo estático que se ha realizado en la fase de análisis se ha realizado el siguiente esquema de las relaciones o tablas necesarias para el sistema:



```
CREATE TABLE Quad (  
    Id INTEGER PRIMARY KEY AUTOINCREMENT,  
    Matricula TEXT NOT NULL UNIQUE,  
    esMonoplaza INTEGER NOT NULL,  
    precio REAL NOT NULL,  
    descripcion TEXT,  
    estaActivo INTEGER NOT NULL DEFAULT 1  
);
```

```
CREATE TABLE Reserva (  
    Id INTEGER PRIMARY KEY AUTOINCREMENT,  
    fecha_recogida TEXT NOT NULL,  
    fecha_devolucion TEXT NOT NULL,  
    num_telefono INTEGER NOT NULL,  
    nombre_cliente TEXT NOT NULL,  
    estaActivo INTEGER NOT NULL DEFAULT 1  
);
```

```
CREATE TABLE Quad_Reserva (  
    Reservaid INTEGER NOT NULL,  
    QuadId INTEGER NOT NULL,  
    num_cascos INTEGER NOT NULL,  
    precio_diario_acordado REAL NOT NULL,  
    PRIMARY KEY (Reservaid, QuadId),  
    FOREIGN KEY (Reservaid) REFERENCES Reserva(Id) ON UPDATE CASCADE ON  
        DELETE CASCADE,  
    FOREIGN KEY (QuadId) REFERENCES Quad(Id) ON UPDATE CASCADE ON DELETE  
        CASCADE  
);
```

8. Conclusiones y Mejoras

Tras la fase de pruebas, se han realizado las siguientes mejoras en el código:

Inclusión de validaciones en los DAOs para evitar entradas nulas.

Implementación de hilos de fondo (Executors) para que las operaciones pesadas de volumen no afecten a la experiencia de usuario.

Añadido de una función de "Limpieza de Memoria" para gestionar el almacenamiento tras realizar tests de estrés.